

Testing Imperative APIs

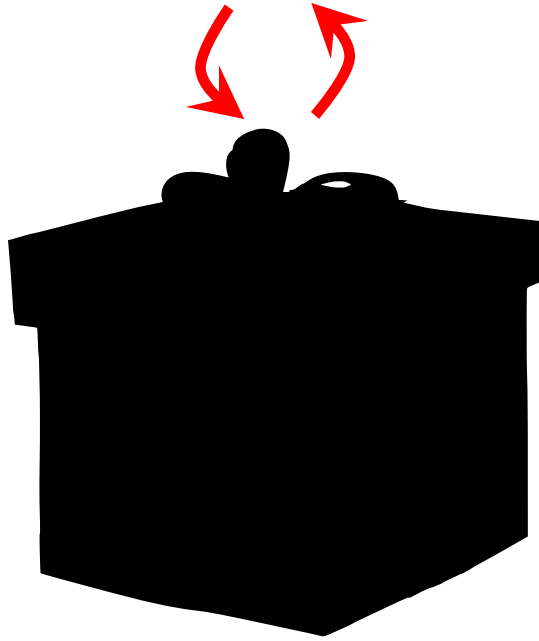
21st International Summer
School on Trustworthy Software

上海

John Hughes



`register (Name, ThreadId)`



`whereis (Name) → ThreadId`

`unregister (Name)`

```
register("a", threadId)
```

???

```
register("a", threadId)  
whereis("a")
```



*Check result is
threadId*

How can we generate a random state?



```
register("a", threadId1)  
register("b", threadId2)  
unregister("a")  
register("c", threadId3)
```

•

•

•

A test case is:
a random sequence of actions

Every action:

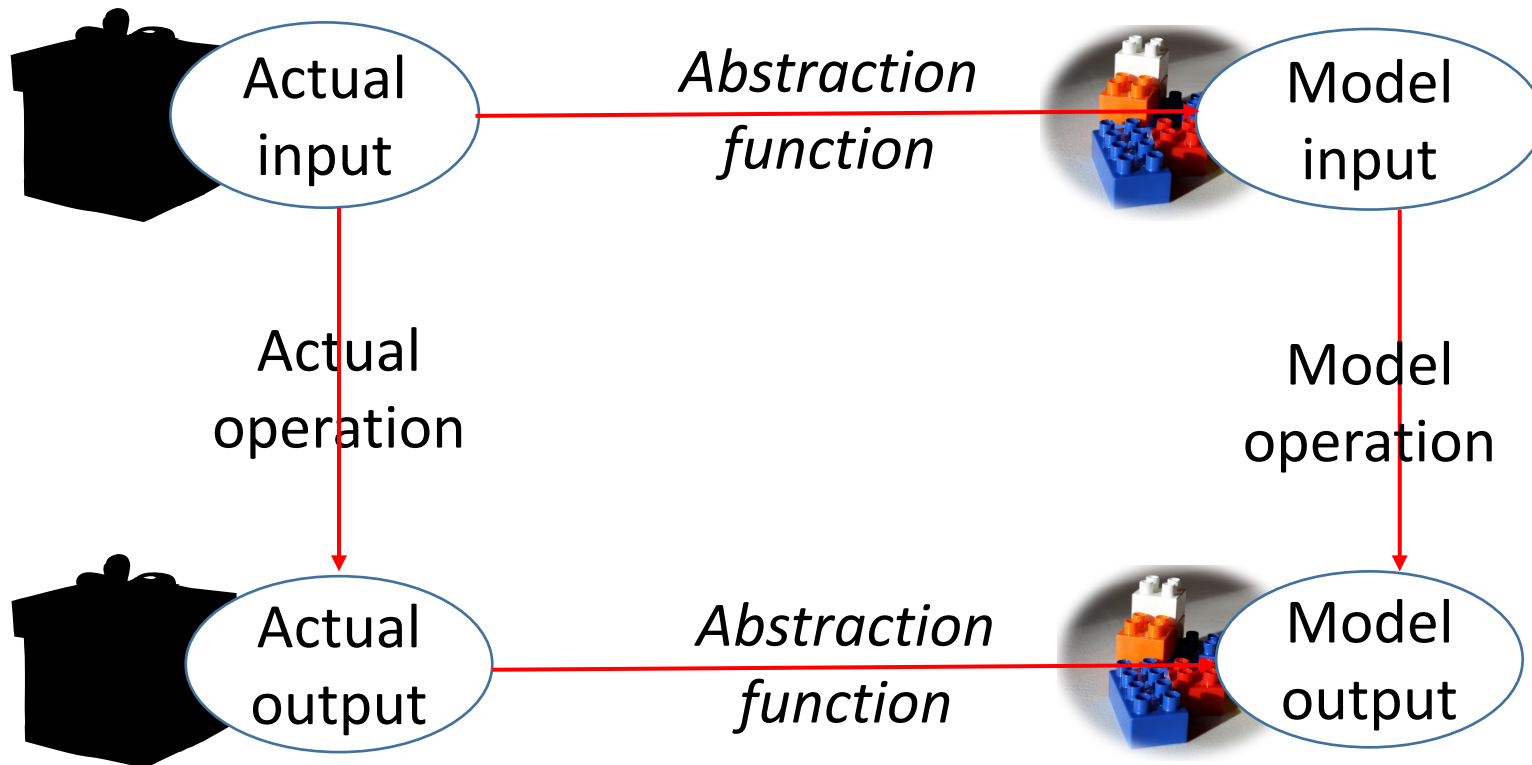
- Contributes to reaching a random state
- Contributes to observing the state

How can we know what is in the state?

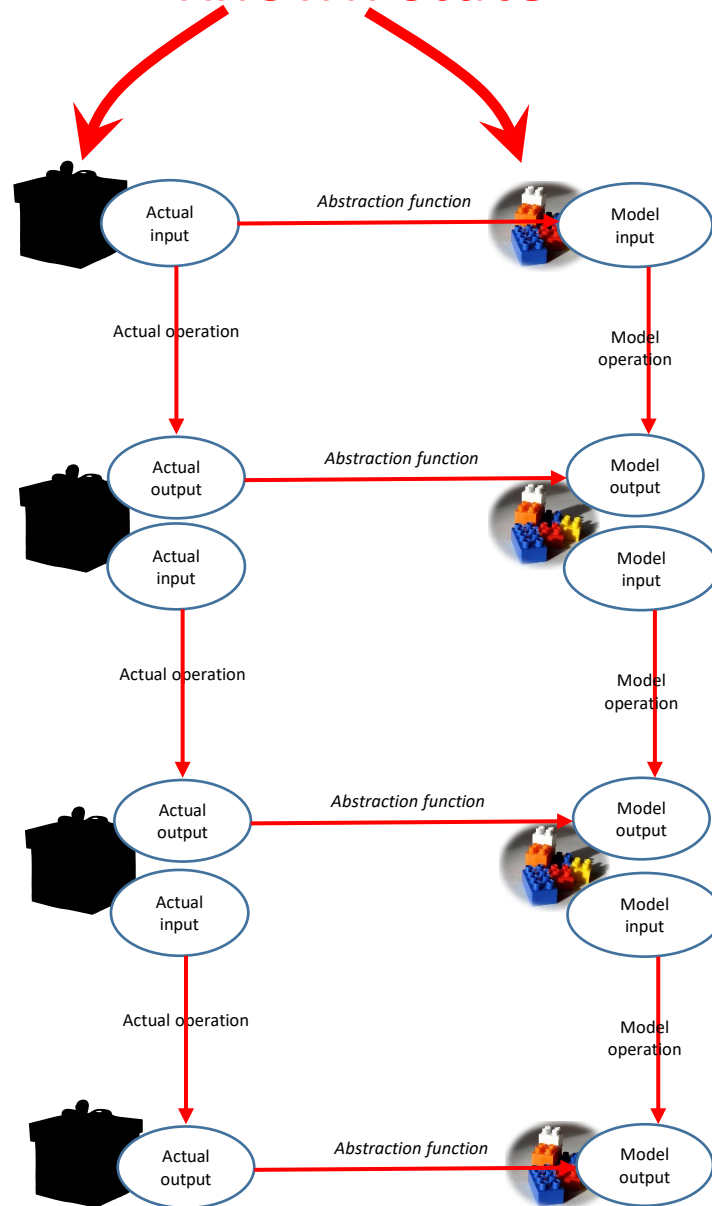


*Keep up-to-date
with model state
transitions*

Monday: Model based tests



Known state



State Modelling Libraries

- Concept of a *state* and an *action*
- The *library* generates, shrinks, and executes the action sequences...
- ...given that the user does the same for the *actions*

State Modelling Libraries

- Quviq QuickCheck **eqc_statem**
- ...
- **Test.StateMachine** in quickcheck-state-machine
- Today: quickcheck-dynamic

Defining Actions

```
data RegState = ...
```



*Overloading:
RegState determines
instance*

```
instance StateModel RegState where
```

```
data Action RegState a where
```

```
  WhereIs :: String ->
```

```
    Action RegState (Maybe ThreadId)
```

```
  Register :: String -> Var ThreadId ->
```

```
    Action RegState ()
```

```
  Unregister :: String ->
```

```
    Action RegState ()
```

```
Spawn :: Action RegState ThreadId
```

Result type

Generating Actions

*All the variables resulting
from previous actions*

```
arbitraryAction ctx _s =  
  let threadCtx = ctxAtType @ThreadId ctx  
  in oneof $  
    [ return $ Some Spawn ]  
  ++ [ Some <$> (Register <$>  
    arbitraryName <*> elements threadCtx)  
    | not . null $ threadCtx ]
```

*The variables of type
ThreadId*

How should names be chosen?

- We *want* the same name to appear repeatedly in the same test case
- Probably the actual strings used is not important

```
allNames = ["a", "b", "c", "d", "e"]
```

```
arbitraryName = elements allNames
```

Now we can generate tests!

```
*RegistryModel1> sample (arbitrary @ (Actions RegState))
do var1 <- action $ Spawn
  action $ Register "a" var1
  action $ Register "c" var1
  pure ()
do action $ Spawn
  pure ()
do pure ()
do pure ()
do var1 <- action $ Spawn
  var2 <- action $ Spawn
  action $ Register "c" var2
  action $ Register "c" var1
  action $ Register "d" var2
  action $ Spawn
...
```

*Specify the **type** of
data to generate*

*Test case (in a format
which can be copied
and pasted into source
code)*

Performing Actions

```
instance RunModel RegState IO where
```

```
  type Error RegState IO = SomeException
```



The type of errors

```
  perform _ Spawn _ = do
    tid <- forkIO (threadDelay 100000000)
    pure $ Right tid
```



10 seconds

```
  perform _ (Register name tid) env = do
    try $ register name (env tid)
```



Catch any exceptions



Look up the variable

The Property

*Actions generated using
the **RegState** functions*



```
prop_Registry :: Actions RegState -> Property
prop_Registry s =
  monadicIO $ do
    runActions s
    QC.assert True
```

```
*RegistryModel1> quickCheck prop_Registry
*** Failed! Falsified (after 9 tests and
    2 shrinks):
do var1 <- action $ Spawn
  action $ Register "c" var1
  pure ()
```

*The failing
test*

```
test1 :: DL RegState ()
test1 =
    do var1 <- action $ Spawn
      action $ Register "c" var1
    pure ()
```

```
*RegistryModel1> quickCheck . withMaxSuccess 1 $  
                        forAllDL test1 prop_Registry  
+++ OK, passed 1 test.
```

...

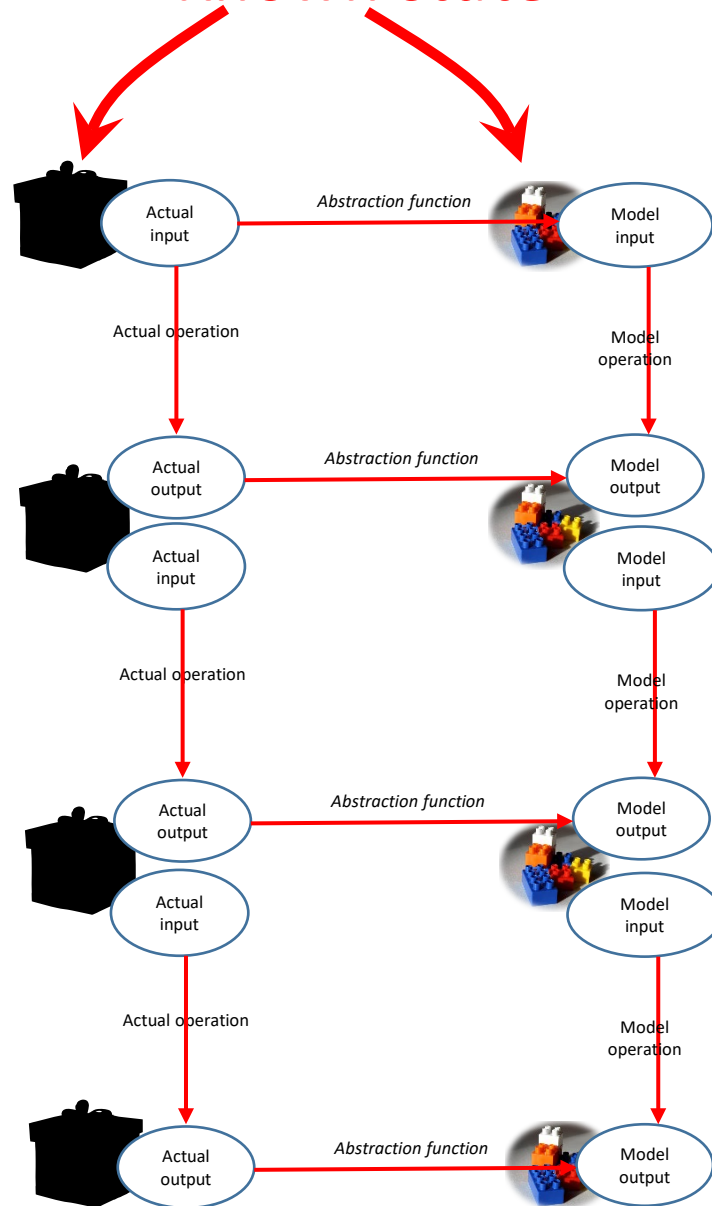
```
*RegistryModel1> quickCheck . withMaxSuccess 1 $  
                        forAllDL test1 prop_Registry  
*** Failed! Falsified (after 1 test):  
do var0 <- action $ Spawn  
  action $ Register "c" var0  
  pure ()
```

```
*RegistryModel1> quickCheck $ forAllDL test1 prop_Registry
*** Failed! Falsified (after 2 tests):
do var0 <- action $ Spawn
  action $ Register "c" var0
pure ()
```

Tests that succeed or fail
at random strongly
suggest *interference*
between tests

**DON'T TRY TO
DEBUG THIS!!!**

Known state



```
prop_Registry :: Actions RegState -> Property
prop_Registry s = monadicIO $ do
    run cleanupRegistry
    runActions s
    QC.assert True
```

```
cleanupRegistry :: IO [Either SomeException ()]
cleanupRegistry =
    sequence [try $ unregister name
              | name <- allNames]
```

```
*RegistryModel1> quickCheck $  
                      forAllDL test1 prop_Registry  
+++ OK, passed 100 tests.
```

```
*RegistryModel1> quickCheck $ prop_Registry
*** Failed! Falsified (after 8 tests and 3
shrinks):
do var4 <- action $ Spawn
  action $ Register "a" var4
  action $ Register "d" var4
  pure ()
```

```
test2 =  
  do var4 <- action $ Spawn  
    action $ Register "a" var4  
    action $ Register "d" var4  
  pure ()
```

```
> quickCheck $ forAllDL test2 prop_Registry  
*** Failed! Falsified (after 1 test):  
do var0 <- action $ Spawn  
  action $ Register "a" var0  
  action $ Register "d" var0  
  pure ()
```

Positive testing

- We test the cases that should work
 - Our tests *should not include* calls that will fail!
 - "Happy path testing"
 - **Advantage:** we test the *interesting* intended behaviour

Negative testing

- We *include* failing calls in our tests
 - We catch exceptions and check that the error behaviour is as it should be
 - **Advantage:** can expose all kinds of dangerous behaviours and vulnerabilities in cases many forget to test

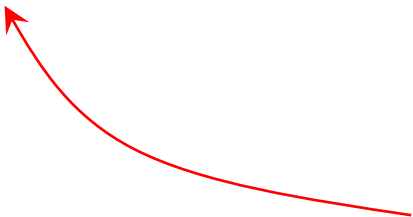
Positive testing

*Define a
precondition*

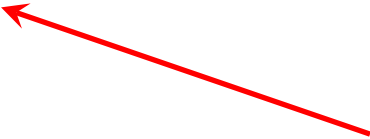


- We *should not* call register twice with the same thread ID
- We need to know *which threads have been registered*

Record it in the model state



```
data RegState = RegState
  { regs :: Map String (Var ThreadId)
  }
```



*Haskell **record**, because we usually need to add fields as we refine the model*

```
initialState = RegState mempty
```

```
nextState s (Register name tid) _ =
  s{regs = Map.insert name tid (regs s)}
nextState s _ _ = s
```

```
precondition s (Register name tid) =
  tid `notElem` Map.elems (regs s)
precondition _ _ = True
```

```
> quickCheck $ forAllDL test2 prop_Registry
*** Failed! Falsified (after 1 test):
do var0 <- action $ Spawn
  action $ Register "a" var0
    -- In state: [var0 :: ThreadId, var1 :: ()] |-
                  RegState {regs = fromList [("a",var0)]}
  action $ Register "d" var0  -- Failed precondition
pure ()
```

```
*RegistryModel1> quickCheck prop_Registry
*** Failed! Falsified (after 10 tests and
4 shrinks):
do var3 <- action $ Spawn
  var4 <- action $ Spawn
  action $ Register "e" var4
  action $ Register "e" var3
  pure ()
```

Strengthen the precondition!

```
precondition s (Register name tid) =  
    name `Map.notMember` regs s  
    && tid `notElem` Map.elems (regs s)  
precondition _ _ = True
```

```
test3 :: DL RegState () test3 =  
  do var3 <- action $ Spawn  
    var4 <- action $ Spawn  
    action $ Register "e" var4  
    action $ Register "e" var3  
  pure ()
```

```
> quickCheck $ forAllDL test3 prop_Registry
*** Failed! Falsified (after 1 test):
do var0 <- action $ Spawn
  var1 <- action $ Spawn
  action $ Register "e" var1
  -- In state: [var0 :: ThreadId,
                var1 :: ThreadId,
                var2 :: ()] |-
                RegState {regs = fromList [("e",var1)]}
  action $ Register "e" var0  -- Failed precondition
pure ()
```

```
*RegistryModel1> quickCheck prop_Registry  
+++ OK, passed 100 tests.
```

...

```
Actions (2641 in total):
```

```
88.83% +Spawn
```

```
11.17% +Register
```

```
Actions rejected by precondition (1873 in total):
```

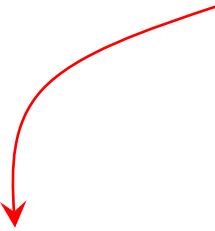
```
100.00% Register
```

Adding **unregister/whereis**

- An exercise for the reader!
- **unregister** requires a precondition (the name must be registered), **whereis** does not.

Checking whereis

*Do we get **Just tid** when **name** is in the registry?*



```
whereis :: String -> IO (Maybe ThreadId)
```

*Do we get the correct **ThreadId**?*



The model state contains all the information needed to check the result of whereis—but there is no code to do so!

In the RunModel class...

*State before
and after*



Values for variables:

Var a -> a



```
postcondition (s, s') (WhereIs name) env result =  
  pure $  
    (env <$> Map.lookup name (regs s)) == result  
        
      Maybe (Var ThreadId)
```

```
postcondition _ _ _ _ = pure True
```

```
*RegistryModel1> quickCheck prop_Registry  
+++ OK, passed 100 tests.
```

...

```
Actions (2927 in total):
```

```
36.69% +Spawn
```

```
35.12% +WhereIs
```

```
16.50% +Register
```

```
11.68% +Unregister
```

```
Actions rejected by precondition (1161 in total):
```

```
60.03% Unregister
```

```
39.97% Register
```

Negative Testing

If a precondition does *not* hold, then the action should

- Raise an exception
- Have no other effect

- In **StateModel** instance:

```
validFailingAction _ _ = True
```

- In **RunModel** instance:

```
postconditionOnFailure _ _ _ res =  
  pure $ isLeft res
```

```
*RegistryModel1> quickCheck prop_Registry  
+++ OK, passed 100 tests.
```

...

```
Action polarity (2712 in total):  
70.02% +  
29.98% -
```

```
Actions (2712 in total):  
26.59% +Spawn  
24.96% +WhereIs  
18.55% -Unregister  
11.50% +Register  
11.43% -Register  
6.97% +Unregister
```

Refinement: crashing threads

- What if threads crash (or terminate?)
- Simulate by adding a KillThread action to tests
- The original documentation said *nothing* about this, so the same model should work, right?

```
data Action RegState a where
```

```
...
```

```
  KillThread :: Var ThreadId -> Action RegState ()
```

```
arbitraryAction ctx _s =
```

```
  let threadIdCtx = ctxAtType @ThreadId ctx  
    in oneof $
```

```
    ...
```

```
    ++ [ Some <$>
```

```
        (KillThread <$> elements threadIdCtx)
```

```
        | not . null $ threadIdCtx ]
```

```
perform _ (KillThread tid) env = do
```

```
  killThread (env tid)
```

```
  threadDelay 100
```

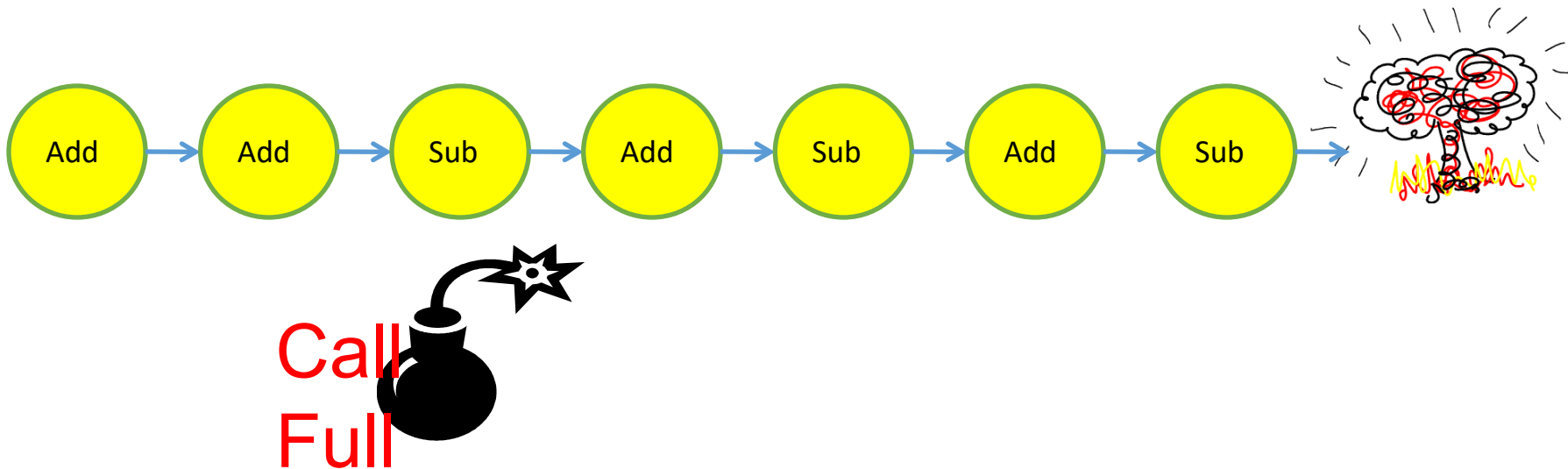
```
  pure $ Right ()
```

```
*RegistryModel1> quickCheck prop_Registry
*** Failed! Falsified (after 3 tests and 1 shrink):
do var2 <- action $ Spawn
  action $ KillThread var2
  action $ Register "b" var2
  pure ()
```

```
*RegistryModel1> quickCheck prop_Registry
*** Failed! Assertion failed (after 9 tests and 5
shrinks):
do var3 <- action $ Spawn
  action $ Register "d" var3
  action $ KillThread var3
  action $ WhereIs "d"
  pure ()
```

Example bug: Media Proxy

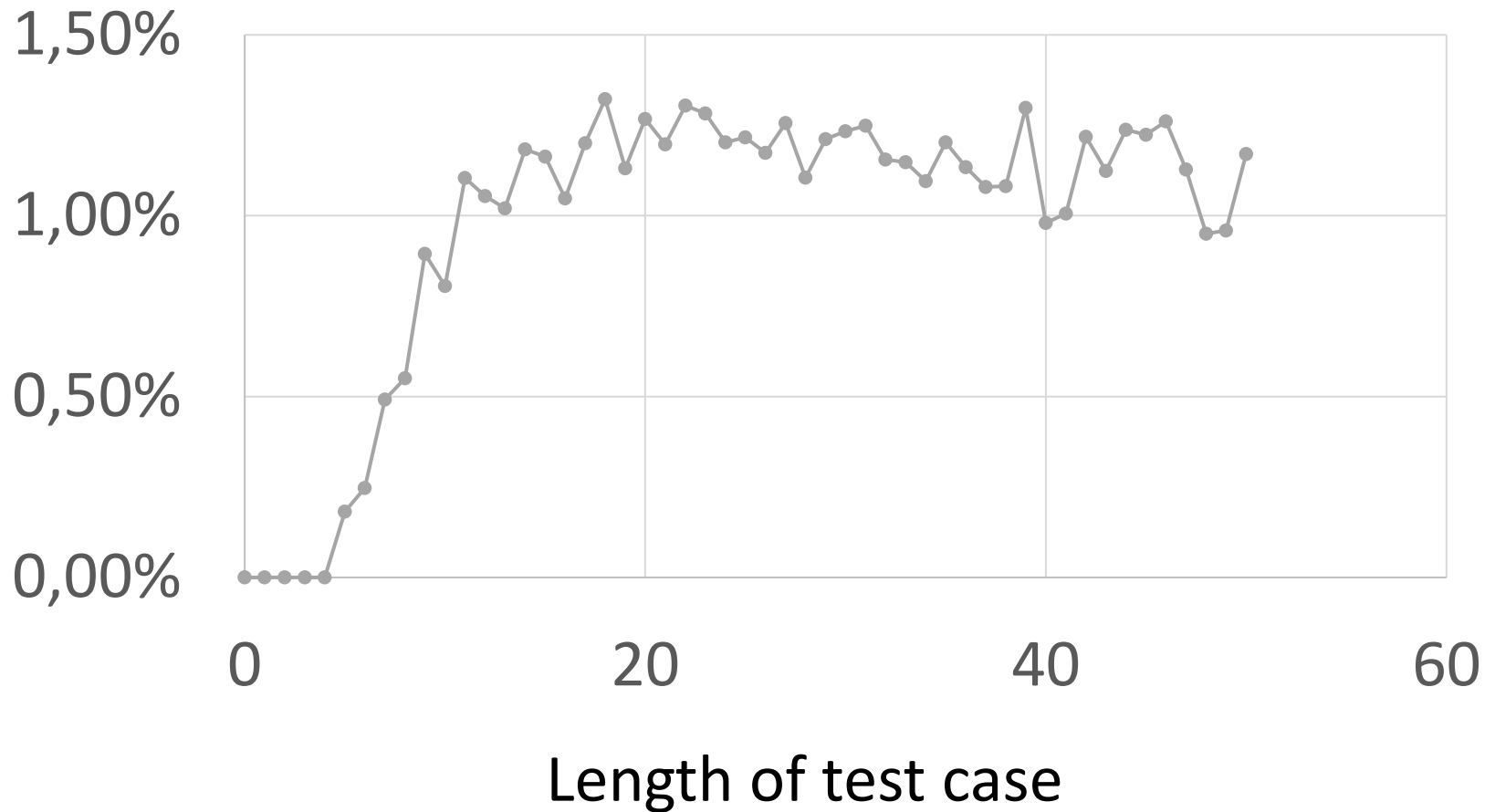
- Multimedia IP-telephony (IMS)
- Connects calls across a firewall
- Test adding and removing callers from a call



Probability of test failure

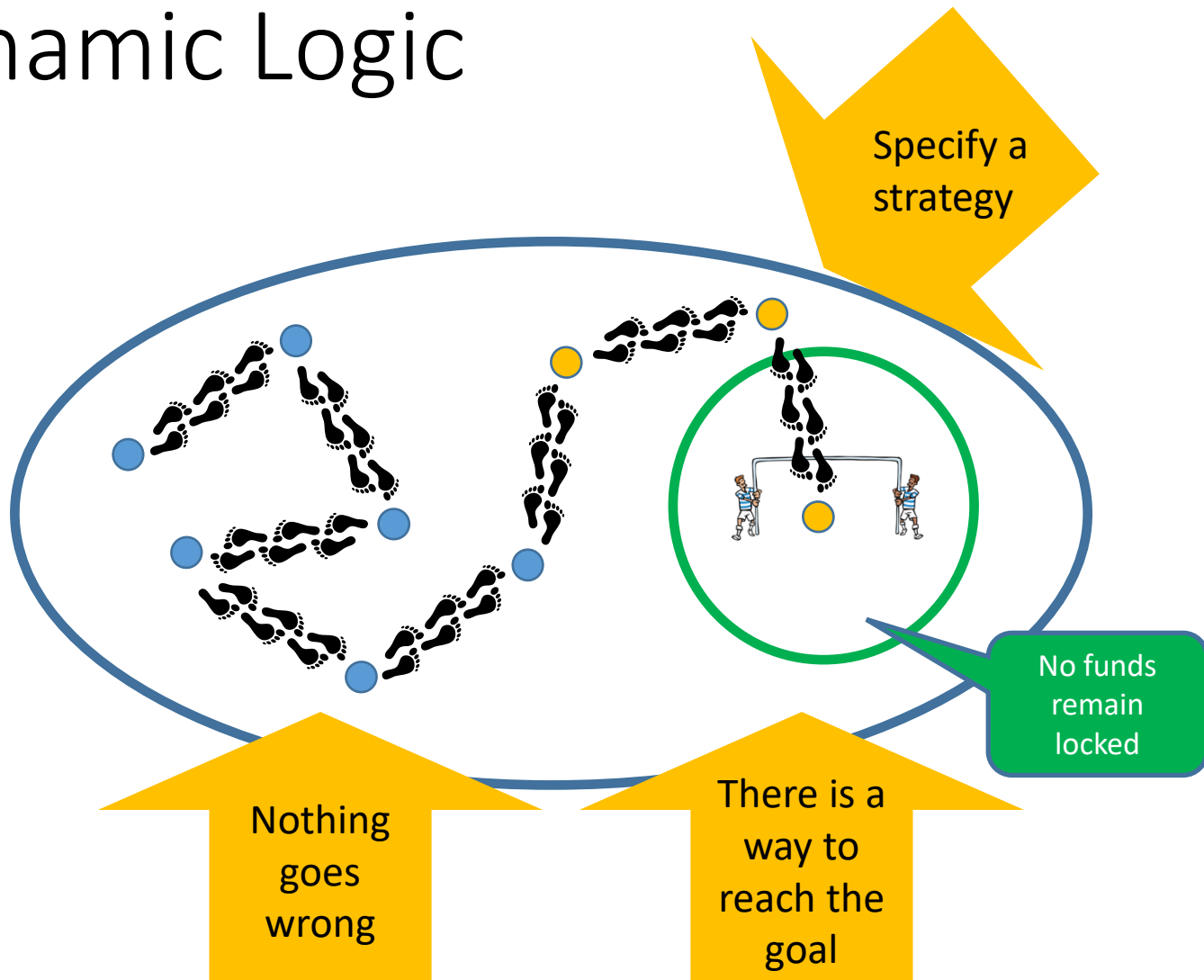


Probability of action failure





Dynamic Logic



Example: you can always register a name and tid

```
canRegister :: DL RegState ()
canRegister = do
  anyActions_
  name <- forAllQ (elementsQ allNames)
  tids <- ctxAtType <$> getVarContextDL
  tid <- forAllQ $ elementsQ tids
  action $ Register name tid
  pure ()
```

```

*RegistryModel1> quickCheck $
    forAllDL canRegister prop_Registry
*** Failed! Falsified (after 5 tests and 1 shrink):
do var2 <- action $ Spawn
  action $ Register "c" var2
  _ <- forAllQ $ exactlyQ $ "a"
  _ <- forAllQ $ exactlyQ $ var2
-- In state: [var2 :: ThreadId, var3 :: ()] |-
--           RegState {regs = fromList [("c",var2)],
--           dead = []}
action $ Register "a" var2 -- Failed precondition
pure ()

```

} anyActions_

} The choices

```
canRegister :: DL RegState ()
canRegister = do
  anyActions_
  name <- forAllQ (elementsQ allNames)
  tids <- ctxAtType <$> getVarContextDL
  tid <- forAllQ $ elementsQ tids
  s <- getModelStateDL
  sequence
    [action $ Unregister name'
    | (name', tid') <- Map.toList (regs s),
      tid' == tid]
  action $ Register name tid
  pure ()
```

```

> quickCheck $ forAllDL canRegister prop_Registry
*** Failed! Falsified (after 7 tests and 3 shrinks):
do var1 <- action $ Spawn
  action $ Register "b" var1
  var5 <- action $ Spawn
  _ <- forAllQ $ exactlyQ $ "b"
  _ <- forAllQ $ exactlyQ $ var5
-- In state: [var1 :: ThreadId, var3 :: (),
--           var5 :: ThreadId] |-
--           RegState {regs = fromList [("b",var1)],
--           dead = []}
action $ Register "b" var5 -- Failed precondition
pure ()

```

Summary of the approach

- Make an abstract model of the system state
- Generate random sequences of actions, possibly using results of previous ones
- Use preconditions to ensure test cases make sense
- Use postconditions to check actual behaviour is consistent with the model

Key takeaways

- Stateful software is harder to test than pure functions, but state-machine models offer an *effective way* to do so.
- Random generation and shrinking is still highly effective, but intricate enough that a good library is essential.
- Stateful software is widespread: most tests used by Quviq customers are of this form.